



1. Problem Title & Link

Title: LeetCode 994 – Rotting Oranges

Link: <https://leetcode.com/problems/rotting-oranges/>

2. Problem Statement (Short Summary)

You are given an $m \times n$ grid where:

- 0 → Empty Cell
- 1 → Fresh Orange
- 2 → Rotten Orange

Every **minute**, every fresh orange adjacent (up, down, left, right) to a rotten orange becomes rotten.

Return:

- Minimum minutes required until all oranges become rotten.
- If impossible, return -1.

3. Examples (Input → Output)

Example 1

Input

grid =

```
[
  [2,1,1],
  [1,1,0],
  [0,1,1]
]
```

Output

4

Example 2

Input

grid =

```
[
  [2,1,1],
  [0,1,1],
  [1,0,1]
]
```

Output



-1

Example 3

Input

grid =

```
[  
  [0,2]  
]
```

Output

0

4. Constraints

- $1 \leq m, n \leq 10$
- $grid[i][j] \in \{0,1,2\}$

5. Core Concept (Pattern / Topic)

- Multi-Source BFS
- Matrix Traversal
- Queue
- Graph Traversal

6. Thought Process (Step-by-Step Explanation)

Observation

Rotting spreads simultaneously.

If there are multiple rotten oranges,
they all spread **at the same time**.

Example

2 . . 2

↓

Both spread together

This is exactly what **Multi-Source BFS** models.

Why DFS Doesn't Work

DFS spreads one orange completely before considering another.



Example

2 . . . 2

DFS would do

Left Rotten

↓

↓

↓

↓

Right Rotten

But in reality

both rotten oranges spread simultaneously.

Therefore,

DFS ❌

BFS ✔

Step 1: Put Every Rotten Orange into Queue

Initially,

all rotten oranges are sources.

Queue

[(0,0), (2,3), ...]

Step 2: Count Fresh Oranges

We need to know when every fresh orange has become rotten.

Maintain

fresh = number of fresh oranges

Step 3: BFS Level by Level

Each BFS level represents

One Minute

Process every rotten orange currently in the queue.



For each,
infect all neighboring fresh oranges.
Those newly rotten oranges become the next BFS level.

Step 4: Stop

After BFS,
If
fresh == 0
return minutes.
Otherwise,
-1
because some oranges were isolated.

Why This Works

BFS naturally processes nodes **level by level**.
Each level equals one minute.
Since every rotten orange starts together,
BFS perfectly simulates simultaneous spreading.

7. Visual / Intuition Diagram (ASCII)

```
Initial
2 1 1

1 1 0

0 1 1
Minute 1
2 2 1

2 1 0

0 1 1
Minute 2
2 2 2

2 2 0

0 1 1
Minute 3
2 2 2

2 2 0
```



```
0 2 1
Minute 4
2 2 2

2 2 0

0 2 2
Answer
4
```

8. Pseudocode (Language Independent)

```
queue = all rotten oranges

count fresh oranges

minutes = 0

while queue not empty AND fresh > 0

    process current queue size

    infect neighbors

    add newly rotten oranges

    minutes++

if fresh == 0

    return minutes

return -1
```

9. Code Implementation

Python

```
from collections import deque

class Solution:

    def orangesRotting(self, grid):

        rows = len(grid)
        cols = len(grid[0])

        queue = deque()
        fresh = 0
```



```
for r in range(rows):
    for c in range(cols):

        if grid[r][c] == 2:
            queue.append((r, c))

        elif grid[r][c] == 1:
            fresh += 1

directions = [(1,0), (-1,0), (0,1), (0,-1)]

minutes = 0

while queue and fresh > 0:

    for _ in range(len(queue)):

        x, y = queue.popleft()

        for dx, dy in directions:

            nx = x + dx
            ny = y + dy

            if (
                0 <= nx < rows and
                0 <= ny < cols and
                grid[nx][ny] == 1
            ):

                grid[nx][ny] = 2
                fresh -= 1
                queue.append((nx, ny))

    minutes += 1

return minutes if fresh == 0 else -1
```

Java

```
import java.util.*;

class Solution {

    public int orangesRotting(int[][] grid) {

        int rows = grid.length;
        int cols = grid[0].length;

        Queue<int[]> queue = new LinkedList<>();
```



```
int fresh = 0;

for(int r = 0; r < rows; r++){

    for(int c = 0; c < cols; c++){

        if(grid[r][c] == 2)
            queue.offer(new int[]{r,c});

        else if(grid[r][c] == 1)
            fresh++;

    }
}

int[][] directions = {
    {1,0},
    {-1,0},
    {0,1},
    {0,-1}
};

int minutes = 0;

while(!queue.isEmpty() && fresh > 0){

    int size = queue.size();

    for(int i = 0; i < size; i++){

        int[] cell = queue.poll();

        for(int[] dir : directions){

            int nr = cell[0] + dir[0];
            int nc = cell[1] + dir[1];

            if(
                nr >= 0 &&
                nr < rows &&
                nc >= 0 &&
                nc < cols &&
                grid[nr][nc] == 1
            ){

                grid[nr][nc] = 2;
                fresh--;
                queue.offer(new int[]{nr,nc});

            }

        }

    }

}
```



```
        minutes++;  
    }  
  
    return fresh == 0 ? minutes : -1;  
}  
}
```

10. Time & Space Complexity

Time Complexity

$O(m \times n)$

Each cell is processed at most once.

Space Complexity

$O(m \times n)$

Queue may contain every orange.

11. Common Mistakes / Edge Cases

- Using DFS instead of BFS.
- Incrementing minutes for every orange instead of every BFS level.
- Forgetting to count fresh oranges initially.
- Forgetting to decrement fresh when an orange becomes rotten.
- Rotting the same orange multiple times.

12. Detailed Dry Run

Initial

2 1 1

1 1 0

0 1 1

Fresh = 6

Queue

[(0,0)]

Minute 1

Rot

(0,1)

(1,0)



Fresh = 4

Queue

[(0,1),(1,0)]

Minute 2

Rot

(0,2)

(1,1)

Fresh = 2

Minute 3

Rot

(2,1)

Fresh = 1

Minute 4

Rot

(2,2)

Fresh = 0

Answer

4

13. Common Use Cases (Real-Life / Interview)

- Disease spread simulation.
- Virus propagation in computer networks.
- Fire spreading through a forest.
- Flood propagation.
- Signal broadcasting from multiple transmitters.

14. Common Traps (Important!)

- Using DFS for simultaneous spread.
- Forgetting that **one BFS level equals one minute**.
- Incrementing minutes even when no fresh oranges remain.
- Revisiting already rotten oranges.
- Ignoring the case where there are no fresh oranges initially (answer is 0).



15. Builds To (Related LeetCode Problems)

- LC 200 – Number of Islands
- LC 695 – Max Area of Island
- LC 286 – Walls and Gates
- LC 542 – 01 Matrix
- LC 1091 – Shortest Path in Binary Matrix
- LC 1971 – Find if Path Exists in Graph
- LC 841 – Keys and Rooms

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Multi-Source BFS	$O(m \times n)$	$O(m \times n)$	Standard and optimal solution
DFS Simulation	✗	✗	Cannot model simultaneous spread correctly
Minute-by-Minute Grid Scan	$O((m \times n)^2)$	$O(1)$	Brute force; repeatedly scans the grid

17. Why This Solution Works (Short Intuition)

Each rotten orange acts as a source of infection. Since all rotten oranges spread simultaneously, we start BFS from **all rotten oranges at once** (multi-source BFS). Each BFS level represents one minute, and newly infected oranges form the next level. This accurately models the rotting process and guarantees the minimum time required.

18. Variations / Follow-Up Questions

1. Return the coordinates that rot at each minute.
2. Allow diagonal infection (8-directional BFS).
3. Compute the time at which each orange becomes rotten.
4. Simulate multiple infection types spreading simultaneously.
5. Find the shortest path from a fresh orange to the nearest rotten orange.